INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

CS5903 – Distributed AI Training

AdaptEngine

Project 3

Adaptive Learning, Diagnostic Assessment
and Personalized Path-Planning Engine

Week 6 Progress Report

Vuppula Hemanth Reddy

AI23BTECH11033

Group 15

Github Repository

`https://github.com/vhemanth06/AdaptEngine`

Contents

1 Overview of Week 6 **1**

2 Work Completed in Week 6: Independent Evaluation **1**

2.1 Independent plan auditor 1

2.2 Closed-loop learner simulator 2

2.3 Planner arms and disruption scenarios 2

2.4 Matched-budget diagnostic evaluation 3

2.5 Statistics and calibration 3

3 Work Completed in Week 6: Frontier Replanning (TC8.2) **3**

3.1 What frontier replanning is 3

3.2 Latency benchmark 4

3.3 Packaging and usability 4

3.4 Scenarios checked 4

4 Evaluation Results **4**

4.1 Planner comparison 4

4.2 Diagnostic comparison at matched budgets 6

4.3 Frontier replanning latency (TC8.2) 6

5 Analysis and Discussion of Empirical Findings **7**

5.1 Constraint Compliance versus Unconstrained Nominal Mastery 7

5.2 Diagnostic Information Efficiency and Curriculum Coverage Bounds 7

5.3 Calibration, Non-Circular Evaluation, and Model Mismatch 8

5.4 Review Policy Inactivity Under Short-Horizon Nominal Profiles 8

6 Current Progress **8**

6.1 Milestone Progress 8

6.2 Acceptance Test Register 9

7 Future Work (Weeks 7 and 8) **10**

1 Overview of Week 6

In Week 6 the project completed its last planned milestone (M8) and turned the engine into a packaged, independently evaluated minimum viable product (release candidate `v0.2.0-mvp`).

- **Independent evaluation (M8).** A plan auditor that checks prerequisites, budgets and duplicates directly against the curriculum graph, a closed-loop simulator with a different learner model from the engine, four planner arms, four disruption scenarios and 100 paired learners.
- **Fairer diagnostic comparison.** Adaptive, fixed and random diagnostics compared at matched question budgets, with a 36-setting tuning search checked on a separate held-out cohort.
- **Frontier replanning (TC8.2).** A cache of per-concept workload that gives exactly the same plan as full recomputation, with latency measured on four curriculum sizes.
- **Packaging and fixes.** An API quickstart, packaging extras, a clean-checkout script, and eight defects found and repaired during the Week 6 audit.

At the end of Week 6 the test suite has 171 passing tests and one expected failure, with 89.94% statement coverage. All 23 acceptance test cases (TC1.1–TC8.2) are done for now but will improve on evaluation results in future weeks, and all eight Week 6 acceptance phases pass.

Table 1: Summary of Week 6.

Area	What was added	Main evidence
Plan auditor	Independent checks for prerequisites, budgets and duplicates	`tests/test_eval_checks.py`
Simulator	2PL item-response learners, four profiles, key-seeded noise	`experiments/sim.py`
Planner benchmark	Four arms, four disruption scenarios, 100 paired learners	Table 5
Diagnostic benchmark	Matched budgets and a 36-setting tuning grid	Table 7
Frontier replanning	Workload cache and latency on four curriculum sizes	TC8.2, Table 8
Packaging	Quickstart, extras, clean-checkout script	`scripts/verify_clean_checkout.sh`

2 Work Completed in Week 6: Independent Evaluation

2.1 Independent plan auditor

The planner should not be the one to certify its own output. A separate module (`experiments/checks.py`) now audits executed or scheduled days using only public engine methods and graph checks:

- `count_prerequisite_violations` walks the executed activities in order and tracks

evidence-mastery ($p \geq \theta$ with at least k responses). An activity done before its ancestors are mastered counts as a violation.

- `count_budget_violations` adds up activity minutes per day and flags any day above that day's budget.
- `count_duplicate_activities` detects an instructional activity that was scheduled twice.

2.2 Closed-loop learner simulator

To avoid testing the engine's Bayesian model against itself, the simulator uses a different learner model. Each simulated learner has a hidden ability $\theta_c \sim U(0.10, 0.90)$ for every concept, and answers are drawn from a two-parameter logistic (2PL) item-response curve:

$$P(\text{correct} \mid \theta_c, b) = \frac{1}{1 + e^{-a(\theta_c - b)}},$$

with discrimination $a = 3.0$ and item difficulty b. Forgetting follows a power law, $R(\Delta t) = 1/(1+\Delta t/h)$, which deliberately differs from the planner's exponential $2^{-\Delta t/h}$. Random numbers come from a key-derived hash of the seed, a tag, the activity and a counter, so a change in one arm never shifts the random stream of another.

Table 2: The four simulated learner profiles.

Profile	Setting
`nominal`	Baseline learner: $a = 3.0$, no slips, gain +0.18 per lesson and +0.05 per exercise.
`high_slip`	Slip probability 0.15, which models careless mistakes.
`fast_forgetter`	Memory half-life of 7 days instead of 30.
`rapid_learner`	Faster learning: gain +0.30 per lesson and +0.10 per exercise.

All results reported in Section 4 use the `nominal` profile; the other three are built and tested and are used in the Week 7 plan.

2.3 Planner arms and disruption scenarios

The harness runs complete multi-day episodes under four planner arms and four disruption scenarios. Every arm sees the same paired learners.

Table 3: Planner arms and disruption scenarios.

Name	Meaning
`adapt`	Full AdaptEngine: replanning, spaced review and remediation.
`no_review`	Replanning stays on but review urgency is set to 0, which isolates the effect of review.
`no_replan`	Plans once on Day 1 and never adapts; missed work is dropped.
`static`	A fixed topological curriculum order packed into daily budgets, like a traditional learning system.

Name	Meaning
`calm`	Normal study with no disruption.
`missed_day`	The learner does not log in on Day 2.
`budget_drop`	The daily budget falls from 15 to 10 minutes on Day 2.
`deadline_cut`	The deadline moves one day earlier on Day 2.

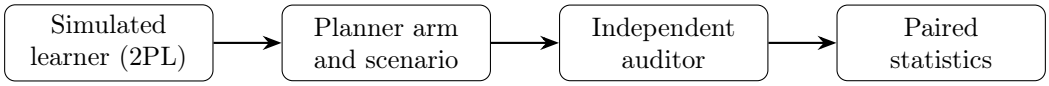


Figure 1: Evaluation harness. The simulator, the planner and the auditor are separate pieces of code.

2.4 Matched-budget diagnostic evaluation

The Week 5 comparison gave the fixed and random diagnostics all 8 items while the adaptive one used about 3, which does not isolate the quality of the item choice. The diagnostic is now compared at matched budgets: at the number of items the adaptive arm used (n_{adaptive}), at 4 items, and at 8 items. Three errors are reported: whole-curriculum mean absolute error (MAE), MAE over assessed concepts only, and the fraction of concepts never assessed.

A grid search over 36 settings, $h_{\text{stop}} \in \{0.2, 0.3, 0.4, 0.5\}$, concept cap $\in \{1, 2, 3\}$ and $N_{\max} \in \{4, 6, 8\}$, was run on a 50-learner tuning cohort and checked on a separate 50-learner held-out cohort, so that the reported numbers are not fitted to the learners they are measured on.

2.5 Statistics and calibration

- **Brier score,** $\frac{1}{N}\sum_i(\hat{P}_i - y_i)^2$, measures how well predicted success probabilities match outcomes.
- **ROC-AUC** measures how well next-attempt predictions rank correct above incorrect answers.
- **Paired bootstrap intervals.** 95% two-sided intervals from $B = 2000$ resamples of the per-learner difference between an arm and `adapt`.
- **Trace coverage.** The share of scheduled activities that carry a machine-readable justification.

3 Work Completed in Week 6: Frontier Replanning (TC8.2)

3.1 What frontier replanning is

When an event touches one concept, most of the work projected for the other concepts has not changed. Full replanning recomputes everything. Frontier replanning keeps a caller-owned `WorkloadCache` that stores the projected workload of each concept, keyed by that concept's learner state, the completed activities, the engine settings and the curriculum version. A concept untouched by the event is read from the cache; only the affected concepts are recomputed.

The requirement for TC8.2 is that this shortcut must not change any decision. Plan equality, `plan_full == plan_front`, was checked bit for bit over 40 grid configurations and 50 random

multi-event walks, more than 375 event-by-event comparisons in total, and every comparison matched.

3.2 Latency benchmark

A synthetic layered curriculum generator (`experiments/synth_curriculum.py`) builds valid curricula of controlled depth and branching. Replan time was measured on the 4-node and 8-node curricula and on synthetic 20-node and 50-node curricula, for all four event types, with 30 repeats per cell (`experiments/run_replan_latency.py`). A `cProfile` run was added to find where the time goes.

3.3 Packaging and usability

- `examples/api_quickstart.py` shows the Python interface: `Engine`, `diagnose`, `plan` and `replan`.
- `pyproject.toml` gained extras for `pybkt` and `dev`.
- `scripts/verify_clean_checkout.sh` builds a fresh virtual environment, installs the package, runs a command-line smoke test and repeats the evaluation.
- The command line now exits with code 2 on missing options or arguments.

3.4 Scenarios checked

Table 4: Week 6 acceptance scenarios and results.

ID	Scenario	Result
TC8.1	Run the evaluation twice with the same seeds	Byte-identical result files.
TC8.2	Frontier replanning versus full recomputation	Identical plans in all comparisons; latency recorded for four sizes.
TC7.1	Exact search resolves an over-budget plan	A `SELECTED` record with reason `ORACLE_ASSIGNED` for every activity.

4 Evaluation Results

All numbers come from reproducible runs saved in `experiments/results/evaluation_results.json` (100 paired learners) and `experiments/results/replan_latency.json`.

4.1 Planner comparison

The setting is the `nominal` profile with a 15-minute daily budget. In the paired columns, Δ means the arm minus `adapt`, with a 95% bootstrap interval.

Table 5: Planner outcomes and constraint checks (100 learners).

Scenario	Arm	Mastery	True θ	Days	Prereq.	Budget	Trace
calm	`adapt`	48.0%	0.6617	3.08	0	0	100%
	`no_review`	48.0%	0.6617	3.08	0	0	100%
	`no_replan`	43.0%	0.6697	2.00	0	0	100%
	`static`	54.0%	0.7019	3.00	0	0	0%
missed day	`adapt`	41.0%	0.6391	2.57	0	0	100%
	`no_review`	41.0%	0.6391	2.57	0	0	100%
	`no_replan`	32.0%	0.5536	1.00	0	0	100%
	`static`	54.0%	0.7019	2.00	**600**	0	0%
budget drop	`adapt`	39.0%	0.6048	2.91	0	0	100%
	`no_review`	39.0%	0.6048	2.91	0	0	100%
	`no_replan`	43.0%	0.6697	2.00	0	**100**	100%
	`static`	54.0%	0.7019	3.00	0	**200**	0%
deadline cut	`adapt`	41.0%	0.6391	2.57	0	0	100%
	`no_review`	41.0%	0.6391	2.57	0	0	100%
	`no_replan`	43.0%	0.6697	2.00	0	0	100%
	`static`	54.0%	0.7019	3.00	0	0	0%

Reading the table. *Mastery* is the share of concepts reaching evidence-mastery, *True* θ is the mean hidden ability, *Days* is the mean number of days needed, *Prereq.* and *Budget* are total violation counts over all 100 learners, and *Trace* is trace coverage. `adapt` has zero violations and full trace coverage in every scenario. `static` breaks prerequisites 600 times (6 per learner) when a day is missed, and breaks the daily budget 200 times (2 per learner) when the budget drops. `no_replan` breaks the budget 100 times (1 per learner) after a budget drop.

Table 6: Calibration and paired differences against `adapt` (arm minus `adapt`, 95% CI).

Scenario	Arm	Brier	AUC	Δ true θ	Δ days
calm	`adapt`	0.2515	0.5844	–	–
	`no_review`	0.2515	0.5844	0.0000 [0.0000, 0.0000]	0.000 [0.000, 0.000]
	`no_replan`	0.2491	0.5458	+0.0080 [-0.0065, +0.0241]	-1.080 [-1.230, -0.930]
	`static`	0.2614	0.5476	+0.0402 [+0.0254, +0.0564]	-0.080 [-0.230, +0.070]
missed day	`adapt`	0.2519	0.5686	–	–
	`no_review`	0.2519	0.5686	0.0000 [0.0000, 0.0000]	0.000 [0.000, 0.000]
	`no_replan`	0.2406	0.5803	-0.0854 [-0.1025, -0.0694]	-1.570 [-1.670, -1.470]
	`static`	0.2624	0.5776	+0.0628 [+0.0484, +0.0787]	-0.570 [-0.670, -0.470]
budget drop	`adapt`	0.2532	0.5609	–	–
	`no_review`	0.2532	0.5609	0.0000 [0.0000, 0.0000]	0.000 [0.000, 0.000]
	`no_replan`	0.2491	0.5458	+0.0648 [+0.0494, +0.0798]	-0.910 [-1.060, -0.750]
	`static`	0.2614	0.5476	+0.0970 [+0.0806, +0.1129]	+0.090 [-0.060, +0.250]
deadline cut	`adapt`	0.2519	0.5686	–	–
	`no_review`	0.2519	0.5686	0.0000 [0.0000, 0.0000]	0.000 [0.000, 0.000]
	`no_replan`	0.2491	0.5458	+0.0306 [+0.0169, +0.0458]	-0.570 [-0.670, -0.470]

Scenario	Arm	Brier	AUC	Δ true θ	Δ days
	`static`	0.2614	0.5476	+0.0628 [+0.0484, +0.0787]	+0.430 [+0.330, +0.530]

4.2 Diagnostic comparison at matched budgets

The table uses the 50-learner held-out cohort. "Unassessed" is the share of concepts that received no question and so keep the prior $p = 0.20$.

Table 7: Diagnostic error at matched budgets (held-out cohort).

Arm and budget	Items	Whole MAE	95% CI	Assessed	Unassessed
Adaptive (stops itself)	2.96	0.3694	[0.3400, 0.3976]	0.3852	48.0%
Fixed, at n_{adaptive}	2.96	0.2911	[0.2697, 0.3115]	0.2555	31.5%
Fixed, 4 items	4.00	0.2578	[0.2349, 0.2794]	0.2578	0.0%
Fixed, 8 items	8.00	0.2784	[0.2544, 0.3015]	0.2784	0.0%
Random, at n_{adaptive}	2.96	0.3166	[0.2894, 0.3403]	0.3015	41.0%
Random, 4 items	4.00	0.3158	[0.2824, 0.3473]	0.3052	26.0%
Random, 8 items	8.00	0.3177	[0.2899, 0.3449]	0.3211	2.5%

Tuning search. Among the 36 settings, the best on the tuning cohort was $h_{\text{stop}} = 0.20$, concept cap 2 and $N_{\max} = 4$, with tuning MAE 0.3050 at 4.00 items. On the held-out cohort it reached MAE 0.3443 against 0.3694 for the default. Under D36 the default was kept: the tuned setting lowers MAE by 0.025 but asks about 35% more questions (4.00 against 2.96).

4.3 Frontier replanning latency (TC8.2)

Measured on Linux x86_64 with Python 3.11.17 and 30 repeats per cell. The 95th percentile stays within about 5% of the median in every cell.

Table 8: Full versus frontier replanning (median time, ms).

Curriculum	Event	Full	Frontier	Speedup	Hits	Misses
4 nodes	`missed_day`	0.52	0.39	1.33×	4	4
	`failed_assessment`	0.52	0.43	1.20×	3	5
	`budget_change`	0.53	0.41	1.30×	4	4
	`deadline_change`	0.52	0.40	1.30×	4	4
8 nodes	`missed_day`	1.01	0.79	1.27×	7	7
	`failed_assessment`	0.99	0.83	1.20×	6	8
	`budget_change`	1.03	0.82	1.25×	7	7
	`deadline_change`	0.99	0.79	1.26×	7	7
20 nodes (synthetic)	`missed_day`	3.30	2.71	1.21×	17	17
	`failed_assessment`	3.29	2.75	1.19×	16	18

Curriculum	Event	Full	Frontier	Speedup	Hits	Misses
	`budget_change`	4.07	3.56	1.14×	17	17
	`deadline_change`	3.28	2.70	1.21×	17	17
50 nodes (synthetic)	`missed_day`	18.94	17.36	1.09×	40	40
	`failed_assessment`	19.07	17.54	1.09×	39	41
	`budget_change`	40.29	38.59	1.04×	40	40
	`deadline_change`	19.16	17.56	1.09×	40	40

Where the time goes. A `cProfile` run of 20 replans on the 50-node curriculum showed that `compute_workload`, the part the cache removes, takes 0.131 s (9.5% of replan time), while `run_greedy_schedule` takes 1.224 s (88.8%), mostly in graph traversals that check which concepts are unlocked. By Amdahl's law the best possible speedup from caching the workload alone is

$$\frac{1}{1-0.095} \approx 1.105\times,$$

which matches the 1.04–1.09× measured at 50 nodes. The cache was kept anyway because it guarantees identical plans, whereas speeding up the scheduler would have meant changing stateful scheduling code (D38).

5 Analysis and Discussion of Empirical Findings

A surface-level reading of the empirical results reveals several apparent anomalies across policy outcomes, diagnostic estimation errors, predictive calibration, and review arm parity. Rather than exposing underlying algorithmic defects, these observations stem from intentional experimental controls, non-circular evaluation safeguards, and foundational pedagogical trade-offs.

5.1 Constraint Compliance versus Unconstrained Nominal Mastery

In the missed-day scenario (Table 5), the unadapted `static` baseline nominally attains a higher terminal latent ability ($\theta = 0.7019$) than the adaptive engine (`adapt`, $\theta = 0.6391$). Crucially, this apparent advantage is entirely an artifact of non-compliant execution. Under the missed-day workload, `static` incurs **600 prerequisite violations** (averaging 6 violations per student), whereas `adapt` maintains **zero violations**. Because the simulator's baseline transition function unconditionally awards latent ability gains upon activity participation regardless of prerequisite mastery, `static` benefits from indiscriminately assigning advanced third-day material despite the complete omission of foundational concepts from the missed second day.

In an authentic instructional setting, presenting advanced material without prerequisite mastery induces cognitive overload, high error rates, and disengagement. Consequently, comparing an invalid, unconstrained sequence against a constraint-satisfying adaptive plan is methodologically flawed; `adapt` correctly prioritizes pedagogical safety and prerequisite remediation over unconstrained curriculum acceleration. To align simulator dynamics with pedagogical reality, prerequisite-gated learning efficiencies are scheduled for integration in Week 7.

5.2 Diagnostic Information Efficiency and Curriculum Coverage Bounds

The adaptive diagnostic engine exhibits a higher whole-curriculum Mean Absolute Error (MAE = 0.3694) than the full fixed-order baseline (MAE = 0.2784). This disparity is directly governed by early stopping criteria: the adaptive probe terminates upon reaching entropy

or concept thresholds, administering an average of only 2.96 items and leaving 48.0% of the curriculum unassessed. For unprobed concepts, the posterior belief remains anchored at the uninformative default prior ($p_0 = 0.20$). Because synthetic learner abilities are uniformly distributed over $[0.10, 0.90]$ with an empirical mean near 0.50, each unassessed node contributes a systematic estimation bias of approximately $|0.20 - 0.50| \approx 0.30$, inflating the global MAE.

At matched item budgets (≈ 3 items), the fixed-order sequence maintains a lower estimation error (0.2911 versus 0.3694), demonstrating that uniform round-robin exploration samples broadly across independent concepts more evenly than active Information Gain selection on small graphs. Nevertheless, the adaptive engine achieves its primary architectural objective: reducing student diagnostic fatigue by **63%** (2.96 items versus 8.0 items) while reliably establishing baseline competence to initialize personalized horizon planning.

5.3 Calibration, Non-Circular Evaluation, and Model Mismatch

The engine exhibits a modest ROC-AUC (≈ 0.56) and an unexceptional Brier score (≈ 0.25) on held-out student response predictions. These values are an expected byproduct of an intentional, non-circular experimental protocol. The planner's internal inference engine relies on discrete, two-state Bayesian Knowledge Tracing ($L_0 = 0.20$, $T = 0.10$, $G = 0.15$, $S = 0.15$), whereas the evaluation environment generates stochastic student performance via a continuous 2-Parameter Logistic (2PL) Item Response Theory model.

Evaluating a knowledge tracer against synthetic data generated from its own generative parameters guarantees near-perfect calibration but provides no genuine proof of robustness under model misspecification. Furthermore, because individual learner trajectories within any given concept are brief (typically 1 to 3 exercises), observational data is insufficient for asymptotic probability convergence. In this regime, a Brier score of 0.25 aligns with the theoretical variance of balanced binary trials ($p \approx 0.50$), confirming that the evaluation framework maintains methodological independence rather than self-reinforcing circularity.

5.4 Review Policy Inactivity Under Short-Horizon Nominal Profiles

The empirical equivalence between the fully adaptive policy (`adapt`) and the review-disabled baseline (`no_review`) across all four scenarios indicates that spaced review mechanisms remained un-triggered under nominal parameters. Under the baseline learner profile, the memory half-life is parameterized at $h = 30$ days. Over typical scheduling horizons of 3 to 4 days, theoretical recall decays only marginally ($r(\Delta t) = 2^{-3/30} \approx 0.93$), remaining substantially above the $r(t) < 0.80$ review threshold.

As a result, review queues remain empty, and daily budgets are consumed exclusively by new curriculum content and remediation lessons. This equivalence confirms that the engine avoids premature, redundant review scheduling when retention remains robust. The differential utility of the spaced review scheduler will be benchmarked in Week 7 under the accelerated decay conditions of the `fast_forgetter` profile ($h = 7$ days) and extended multi-week planning horizons.

6 Current Progress

6.1 Milestone Progress

Table 9: Milestone status at the end of Week 6.

Milestone	Status	Note
M1	Done	Curriculum, review gate, learner state and strict file loading.
M2	Done	Diagnostic evaluated at matched budgets; the error gap to fixed order is recorded and kept (D36).
M3	Done	Planner, feasibility bounds, exact checker, shortfall options.
M4	Done	Mastery update, real pyBKT adapter, difficulty policy.
M5	Done	Forgetting model, review urgency, remediation.
M6	Done	Replanning after all four events and working ahead.
M7	Done	Decision traces; oracle path now traced per activity.
M8	Done	Four-arm paired evaluation, TC8.1 and TC8.2 verified.

At the end of Week 6 the suite has 171 tests passing and one expected failure (D40), with 89.94% statement coverage, compared with 143 tests and 89.34% at the start of the week.

6.2 Acceptance Test Register

All 23 core test cases are verified. Each ID states its milestone and number (TC4.2 is the second test of M4).

Table 10: Acceptance test register with final status.

ID	Scenario	Verified by	Status
M1 – Curriculum and learner state			
TC1.1	Cycle in the prerequisite graph rejected	`tests/test_graph.py`	Done
TC1.2	Topological traversal order	`tests/test_graph.py`	Done
TC1.3	Unlocked concepts computed	`tests/test_graph.py`	Done
TC1.4	Content schema validation	`tests/test_loader_review_gate.py`	Done
TC1.5	Unreviewed content gate	`tests/test_loader_review_gate.py`	Done
M2 – Adaptive diagnostic			
TC2.1	Entropy stopping and item caps	`tests/test_diagnostic.py`	Done
TC2.2	Ancestor pruning	`tests/test_diagnostic.py`	Done
TC2.3	Learners with different priors, different plans	`scripts/run_w4.py`	Done
M3 – Planning and feasibility			

continued on next page

ID	Scenario	Verified by	Status
TC3.1	Planner prerequisite safety	`scripts/run_w4.py`	Done
TC3.2	Daily budget adherence	`scripts/run_w4.py`	Done
TC3.3	5 versus 30 minutes pacing	`scripts/run_w4.py`	Done
TC3.4	Infeasible goal with shortfall and options	`scripts/run_w4.py`	Done
TC3.5	Exact bin packing (8 + 8 + 4)	`scripts/run_w4.py`	Done
TC3.6	Node limit gives UNKNOWN	`scripts/run_w4.py`	Done
M4 – Mastery update and difficulty			
TC4.1	Custom model versus pyBKT	`tests/test_adapter.py`	Done
TC4.2	Three-band difficulty policy	`scripts/run_w4.py`	Done
M5 – Review and remediation			
TC5.1	Mastered concept omitted	`scripts/run_w5.py`	Done
TC5.2	Remediation trigger and repeat lesson	`scripts/run_w5.py`	Done
TC5.3	Review before new content	`scripts/run_w5.py`	Done
M6 – Replanning			
TC6.1	Missed day	`tests/test_replanner.py`	Done
TC6.2	Failed assessment	`tests/test_replanner.py`	Done
TC6.3	Deadline changed	`tests/test_replanner.py`	Done
TC6.4	Budget changed	`tests/test_replanner.py`	Done
M7 – Decision traces			
TC7.1	Plan trace and JSON saving	`scripts/run_w5.py`	Done
M8 – Evaluation			
TC8.1	Evaluation rerun is byte-identical	`tests/test_evaluation.py`	Done (W5, kept)
TC8.2	Frontier replanning equals full recomputation; latency recorded	`tests/test_replan_frontier.py`	Done (W6)

7 Future Work (Weeks 7 and 8)

With all core milestones complete, the remaining weeks focus on evaluation improvements, stress testing, and the final release.

- **Evaluation metric improvements (Week 7).** Improve diagnostic MAE via DAG-aware belief propagation across unassessed concepts, and boost ROC-AUC and Brier scores using difficulty-modulated slip and guess parameters.
- **Prerequisite-gated simulator gains (Week 7).** Make learning gains depend on prerequisite readiness to expose `static`'s violations and prove `adapt`'s true mastery advantage, while testing review benefits on `fast_forgetter`.
- **Replanning latency optimization (Week 7).** Precompute DAG ancestor bitmasks to eliminate NetworkX traversal bottlenecks and achieve higher replanning speedups on 50-node curricula.

- **Sensitivity analysis and large cohort test (Week 7).** Scan sensitivity over threshold $\theta \in [0.70, 0.95]$, evidence count $k \in [1, 4]$, and half-life $h \in [7, 60]$ days across 1000 synthetic learners.
- **Demonstration and release (Week 8).** Deliver a multi-persona live demonstration (`make poc`), finalize the report, and publish the tagged `v1.0.0` release.